Manual de técnico Simulador de Crédito Educativo UPB

Juan David Ordoñez Ferreira

David Eduardo Flórez Duran

David Ricardo Carvajal Barragan

Gabriel Arturo Sastoque Cadena

Gabriel Matias Rodríguez Castro

José David Mora Patiño

Juan Pablo Arenas Avendaño

Santiago Cifuentes Cardenas

Simón Peréz Gómez



Manual presentado para documentación del

Simulador de Crédito Educativo UPB

Universidad Pontificia Bolivariana

Escuela de Ingenierías

Ingeniería de Sistemas e Informática

Bucaramanga

2026

Manual de técnico Simulador de Crédito Educativo UPB

Juan David Ordoñez Ferreira

David Eduardo Flórez Duran

David Ricardo Carvajal Barragan

Gabriel Arturo Sastoque Cadena

Gabriel Matias Rodríguez Castro

José David Mora Patiño

Juan Pablo Arenas Avendaño

Santiago Cifuentes Cardenas

Simón Peréz Gómez

Manual presentado para documentación del

Simulador de Crédito Educativo UPB

Director

Ing. Javier Enrique Lizarazo Rueda

Universidad Pontificia Bolivariana

Escuela de Ingenierías

Ingeniería de Sistemas e Informática

Bucaramanga

2026

Contenido

Control del documento **8**

Propósito y Alcance **9**

Descripción general de la aplicación **10**

Tecnologías y versiones **12**

Arquitectura **13**

Estructura del repositorio **15**

Frontend **16**

Backend **19**

Motor financiero **20**

Modelo de datos **22**

API y contratos **24**

Seguridad **25**

Ambientes y variables **26**

Pruebas y calidad **27**

Despliegue y operación **29**

Mantenimiento **30**

Riesgos y recomendaciones **31**

Glosario **32**

Referencias **33**

Contenido Figuras

Figura 1 10

Figura 2 11

Figura 3 14

Figura 4 17

Figura 5 Historial de simulaciones del estudiante. 17

Figura 6 Panel administrativo para parametrización financiera. 17

Figura 7 21

Figura 8 22

Figura 9 27

Universidad Pontificia Bolivariana
SECCIONAL BUCARAMANGA

Resumen general del manual en español

TITULO: Manual de técnico Simulador de Crédito Educativo UPB

AUTOR(ES): Juan David Ordoñez Ferreira, David Ricardo Carvajal Barragan, David Eduardo Floréz Duran, Gabriel Arturo Sastoque Cadena, Gabriel Matias Rodríguez Castro, José David Mora Patiño, Juan Pablo Arenas Avendaño, Santiago Cifuentes Cardenas, Simón Peréz Gómez

PROGRAMA: Ingeniería de Sistemas e Informática

DIRECTOR(A): Javier Enrique Lizarazo Rueda

RESUMEN

Este manual técnico documenta la aplicación Simulador de Crédito UPB desde una perspectiva de arquitectura, construcción, operación y mantenimiento. La aplicación permite a estudiantes, invitados y administradores simular planes de financiación educativa, consultar historial, exportar resultados y administrar parámetros financieros. El sistema fue desarrollado como monorepo con frontend React/Vite y backend Node.js/Express, aplicando TypeScript, Clean Architecture, Strategy Pattern, validación con Zod, persistencia relacional con Sequelize y MySQL/MariaDB, autenticación local/LDAP, seguridad por capas y pruebas unitarias e integrales.

El documento consolida el estado técnico del proyecto, sus módulos, dependencias, estructura de carpetas, modelo de datos, API, reglas financieras, despliegue, variables de entorno, pruebas, seguridad, riesgos y recomendaciones. El contenido se complementa con evidencias visuales internas, diagramas del repositorio y referencias bibliográficas reales de documentación oficial.

PALABRAS CLAVE:

Arquitectura de software, financiación educativa, simulación financiera, monorepo, React, Vite, Node.js, Express, TypeScript, Clean Architecture, Strategy Pattern, Sequelize, MariaDB, autenticación LDAP

V° B° DIRECTOR DE TRABAJO DE GRADO

Universidad Pontificia Bolivariana
SECCIONAL BUCARAMANGA

General summary of the manual

TITLE: Technical Manual: UPB Educational Credit Simulator

AUTHOR(S): Juan David Ordoñez Ferreira, David Ricardo Carvajal Barragan, David Eduardo Floréz Duran, Gabriel Arturo Sastoque Cadena, Gabriel Matias Rodríguez Castro, José David Mora Patiño, Juan Pablo Arenas Avendaño, Santiago Cifuentes Cardenas, Simón Peréz Gómez

FACULTY: Systems and Computing Engineering

DIRECTOR: Javier Enrique Lizarazo Rueda

ABSTRACT

This technical manual documents the Simulador de Crédito UPB (UPB Credit Simulator) application from an architectural, construction, operational, and maintenance perspective. The application allows students, guests, and administrators to simulate educational financing plans, consult history, export results, and manage financial parameters. The system was developed as a monorepo featuring a React/Vite frontend and a Node.js/Express backend, applying TypeScript, Clean Architecture, the Strategy Pattern, Zod validation, relational persistence with Sequelize and MySQL/MariaDB, local/LDAP authentication, layered security, and both unit and integration testing.

This document consolidates the technical state of the project, including its modules, dependencies, folder structure, data model, API, financial rules, deployment, environment variables, testing, security, risks, and recommendations. The content is complemented by internal visual evidence, repository diagrams, and real bibliographical references from official documentation.

KEYWORDS:

Software architecture, educational financing, financial simulation, monorepo, React, Vite, Node.js, Express, TypeScript, Clean Architecture, Strategy Pattern, Sequelize, MariaDB, LDAP authentication

V° B° DIRECTOR OF GRADUATE WORK

Control del documento

Campo	Valor
Nombre	Manual técnico de la aplicación desarrollada
Sistema	Simulador de Crédito UPB
Versión del manual	1.0
Fecha	12 de mayo de 2026
Audiencia	Docentes, jurados, CTIC, desarrolladores y personal técnico de soporte

Propósito y Alcance

El propósito del manual es servir como documento técnico maestro para comprender, operar y mantener el Simulador de Crédito UPB. Está orientado a personas que deban evaluar el proyecto, continuar su desarrollo, instalarlo en infraestructura institucional o diagnosticar fallas. No reemplaza el manual funcional de usuario ni el manual de despliegue CTIC, pero los complementa con la explicación técnica integral de la solución.

Alcance principal
El manual cubre el sistema implementado en el repositorio local: frontend React, backend Express, base de datos relacional, autenticación, simulación financiera, exportación PDF/CSV, panel administrativo, pruebas y despliegue.

Descripción general de la aplicación

El Simulador de Crédito UPB es una plataforma web para calcular escenarios de financiación educativa. El usuario puede operar como invitado para obtener una vista previa sin registro, como estudiante autenticado para guardar y consultar simulaciones, o como administrador para revisar usuarios, simulaciones, parámetros financieros y evidencias de operación.

Figura 1

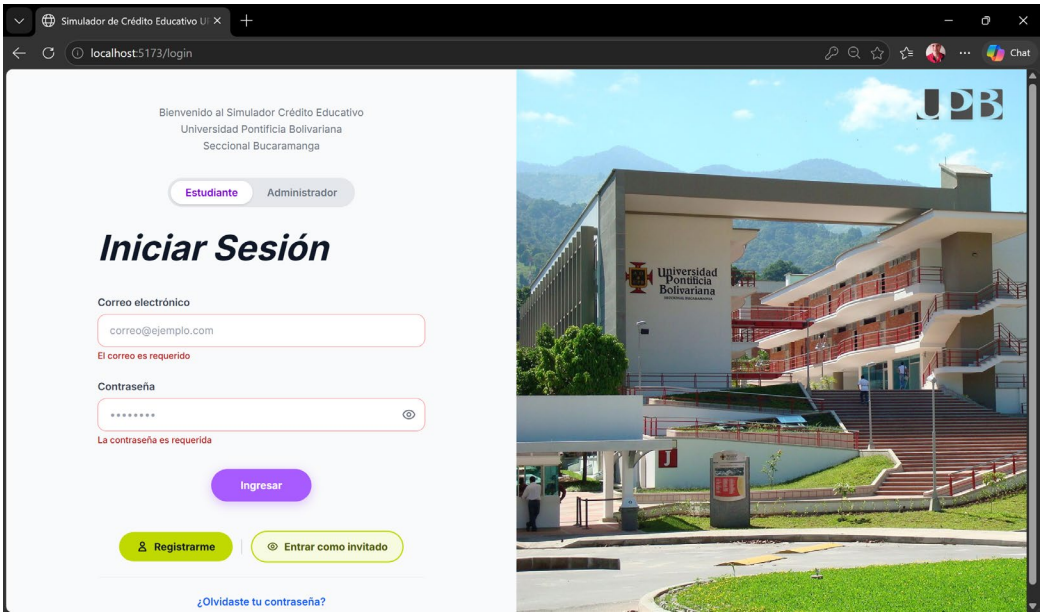
Pantalla inicial del Simulador de Crédito UPB tomada de las evidencias funcionales del repositorio



Actor	Capacidades principales
Invitado	Simular crédito sin autenticarse, revisar resultados y explorar la aplicación sin historial persistente.
Estudiante	Registrarse, iniciar sesión, simular, guardar, consultar historial, exportar PDF/CSV y actualizar perfil.
Administrador	Autenticarse con rol ADMIN, gestionar estudiantes, revisar simulaciones, administrar parámetros y auditar operación.
Sistema	Ejecutar jobs de purga, registrar auditoría, validar seguridad y sostener integridad de datos.

Figura 2

Pantalla de autenticación usada en validaciones funcionales.



Tecnologías y versiones

El proyecto usa tecnologías de ecosistema JavaScript/TypeScript tanto en cliente como en servidor. Las referencias oficiales de React, Vite, TypeScript, Node.js y Express se usan como soporte bibliográfico para las decisiones de stack (Express, n.d.; Meta Open Source, n.d.; Microsoft, n.d.; Node.js Contributors, n.d.; Vite, n.d.).

Capa	**Tecnología**	**Versión en package.json**	**Uso**
Monorepo	npm workspaces	npm	Organiza client y server en un solo repositorio.
Frontend	React	19.2.5	Interfaz de usuario basada en componentes.
Frontend	Vite	8.0.10	Dev server y build del cliente.
Frontend	TypeScript	6.0.3	Tipado estático.
Frontend	Tailwind CSS	4.2.2	Estilos utility-first.
Frontend	React Router	7.14.2	Rutas públicas, privadas y por rol.
Frontend	React Hook Form + Zod	7.74.0 / 4.3.6	Formularios y validación.
Backend	Node.js	20+ recomendado	Runtime del servidor.
Backend	Express	5.2.1	API REST y middlewares HTTP.
Backend	Sequelize + mysql2	6.37.8 / 3.22.3	ORM y driver MySQL/MariaDB.
Backend	JWT + bcrypt	9.0.3 / 6.0.0	Autenticación y hash de contraseñas.
Backend	Nodemailer	8.0.7	Correos transaccionales.
Backend	PDFKit	0.18.0	Generación de reportes PDF.
Pruebas	Jest / Vitest	30.3.0 / 4.1.5	Pruebas backend y frontend.
Producción	PM2 + Nginx	Global / SO	Proceso persistente y reverse proxy.

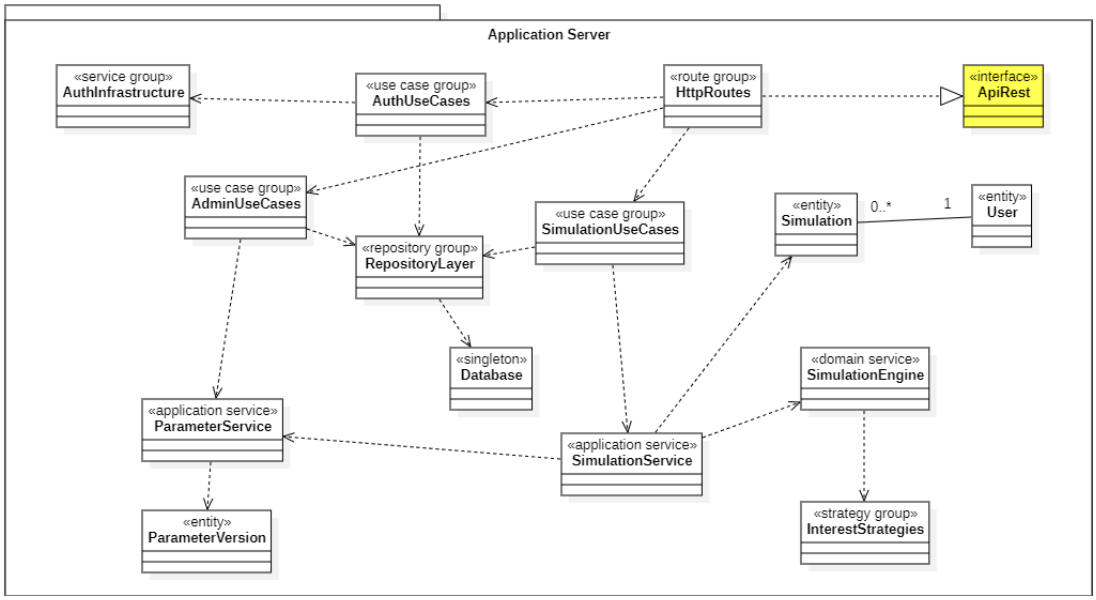
Arquitectura

El backend aplica Clean Architecture con separación entre dominio, aplicación e infraestructura. Esta decisión permite aislar el motor financiero, probar reglas sin depender de Express o MySQL y cambiar detalles externos con menor impacto. El patrón principal del cálculo financiero es Strategy Pattern, usado para separar modalidad sin interés y modalidad con interés.

Capa	**Ruta**	**Responsabilidad**
Core	server/src/Core	Entidades, value objects, errores y contratos de repositorio.
Application	server/src/Application	Casos de uso, servicios, DTOs, motor financiero, estrategias y exportadores.
Infrastructure	server/src/Infrastructure	Express, rutas, middlewares, Sequelize, JWT, LDAP, email, logger y schedulers.
Frontend	client/src	Rutas, páginas, componentes, servicios API y utilidades compartidas.
Documentación	docs	Requisitos, arquitectura, pruebas, despliegue, base de datos y evidencias.

Figura 3

Diagrama de clases del proyecto almacenado en docs/diagrams/class-diagram.



Decisiones técnicas adoptadas
Clean Architecture, Strategy Pattern, validación server-side con Zod, autenticación dual JWT/LDAP, npm workspaces, React/Vite, Sequelize ORM, soft-delete con purga automática y manejo fail-fast de secretos.

Estructura del repositorio

```
Simulador-Credito-UPB/
  client/                 Frontend React + Vite + TypeScript
  server/                 Backend Express + TypeScript
  docs/                   Documentación técnica, pruebas,
despliegue y requisitos
  infrastructure/         Configuración base de
infraestructura
  public/                 Recursos públicos generales
  package.json            Workspaces y scripts raíz
  .env.example            Plantilla de variables sin
secretos reales
```

Script raíz	**Descripción**
npm run dev	Levanta frontend y backend en paralelo.
npm run dev:server	Ejecuta el backend en modo desarrollo.
npm run dev:client	Ejecuta el frontend Vite.
npm run build:server	Compila TypeScript del backend hacia server/dist.
npm run build:client	Compila TypeScript y genera client/dist.

Frontend

El frontend es una Single Page Application con rutas públicas y protegidas. La comunicación con el backend se centraliza en client/src/shared/services/api.ts, usando VITE_API_URL con valor recomendado /api en producción. El estado de autenticación se maneja en AuthContext y las rutas se protegen con ProtectedRoute y PublicRoute.

Módulo frontend	**Rutas / páginas**	**Responsabilidad**
landing	LandingPage, TermsPage	Entrada pública, navegación inicial y términos.
auth	Login, Register, ForgotPassword, ResetPassword	Autenticación, registro y recuperación de contraseña.
simulator/guest	GuestMainPage, GuestCreateSimulationPage	Simulación sin autenticación.
simulator/user	CreateSimulationPage, MainStudentPage, StudentHistoryPage	Simulación autenticada, historial y perfil.
admin	AdminDashboard, AdminStudents, AdminParameters, AdminSimulations	Panel administrativo y parametrización.
shared	Layout, Headers, SimulationForm, charts, utils	Componentes y servicios reutilizables.

Figura 4

Resultado de simulación en modo invitado, evidencia funcional interna.

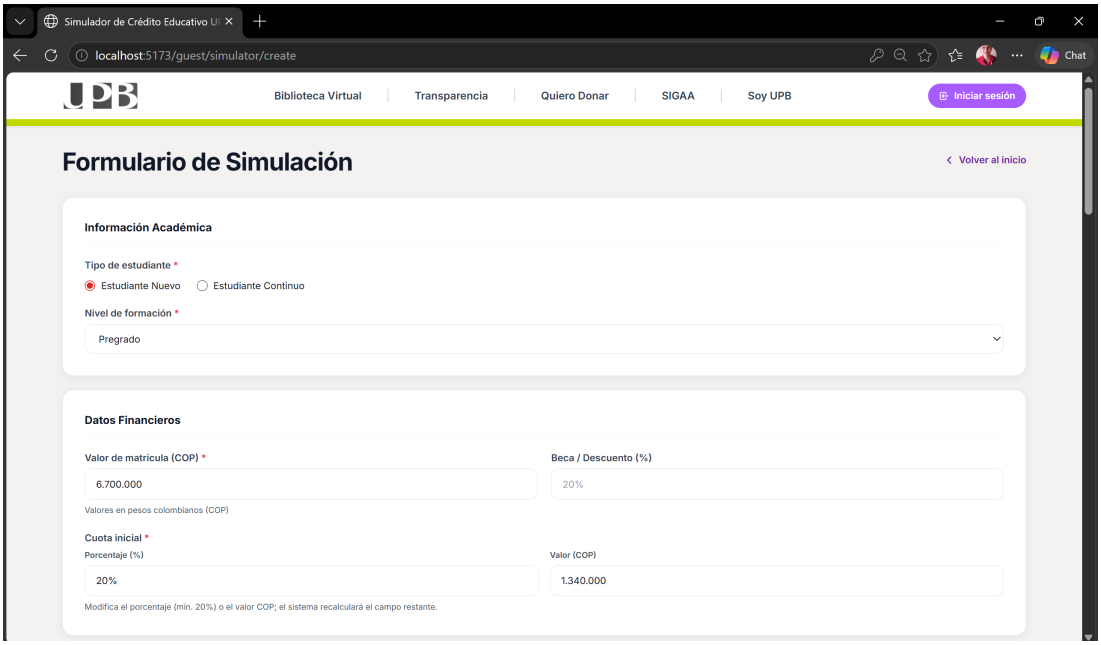


Figura 5

Historial de simulaciones del estudiante.

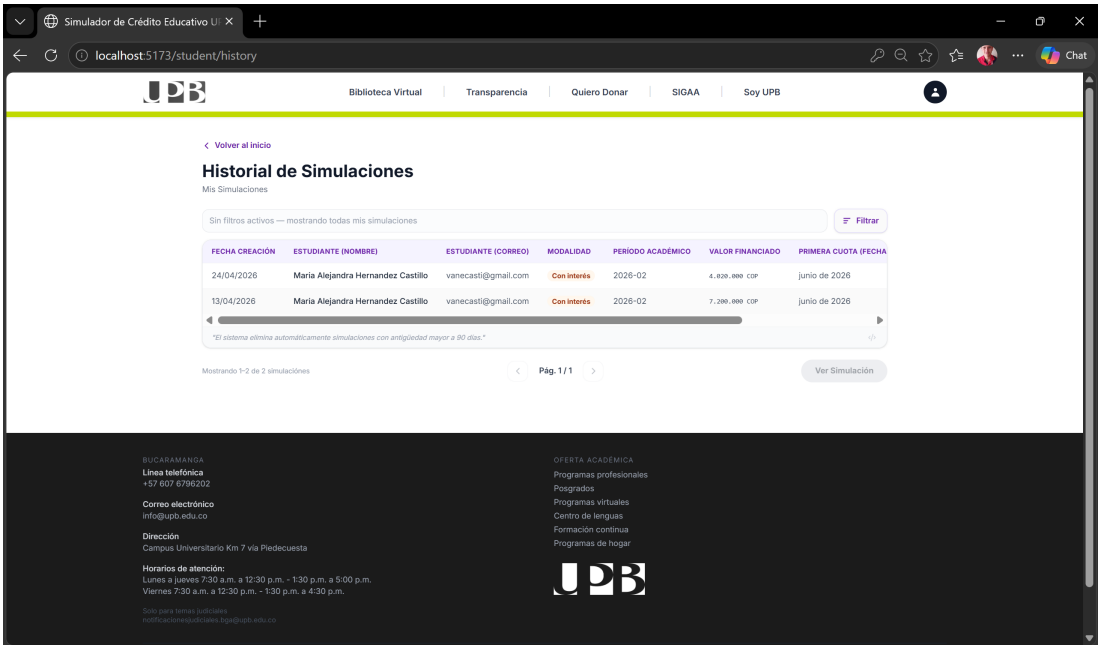
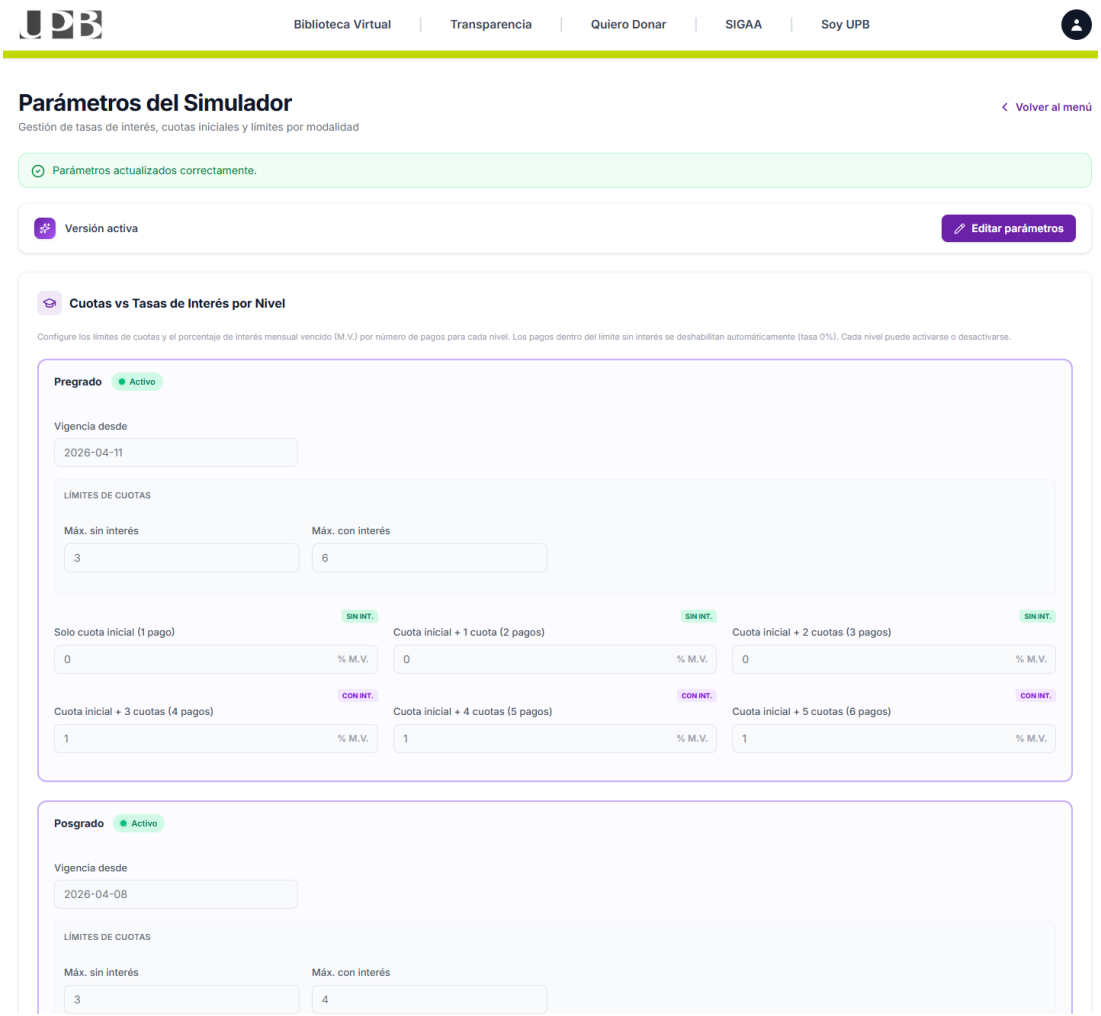


Figura 6

Panel administrativo para parametrización financiera.



Backend

El backend expone una API REST sobre Express 5. La aplicación se crea en Infrastructure/http/app.ts, donde se registran middlewares globales, Swagger, health check, rutas públicas y rutas protegidas. El arranque ocurre en server/src/index.ts, con validación de variables, conexión a base de datos, schedulers y listener HTTP.

Componente backend	**Ruta principal**	**Función**
Rutas HTTP	Infrastructure/http/routes	Define endpoints auth, simulator, admin, users y parámetros públicos.
Middlewares	Infrastructure/http/middlewares	Autenticación, autorización, validación, XSS, rate limit y errores.
Persistencia	Infrastructure/persistence	Modelos y repositorios Sequelize.
Auth	Infrastructure/auth	JWT y LDAP.
Email	Infrastructure/email	Nodemailer para recuperación de contraseña.
Scheduler	Infrastructure/scheduler	Purga de simulaciones y tokens vencidos.
Logger	Infrastructure/logger	Winston y trazabilidad operativa.

Endpoint público	**Método**	**Propósito**
/api/health	GET	Health check del backend.
/api/docs	GET	Swagger UI.
/api/auth/login	POST	Inicio de sesión.
/api/auth/register	POST	Registro de estudiante.
/api/auth/forgot-password	POST	Recuperación por correo.
/api/simulator/preview	POST	Simulación de invitado o vista previa.
/api/parameters/active	GET	Parámetros financieros activos.

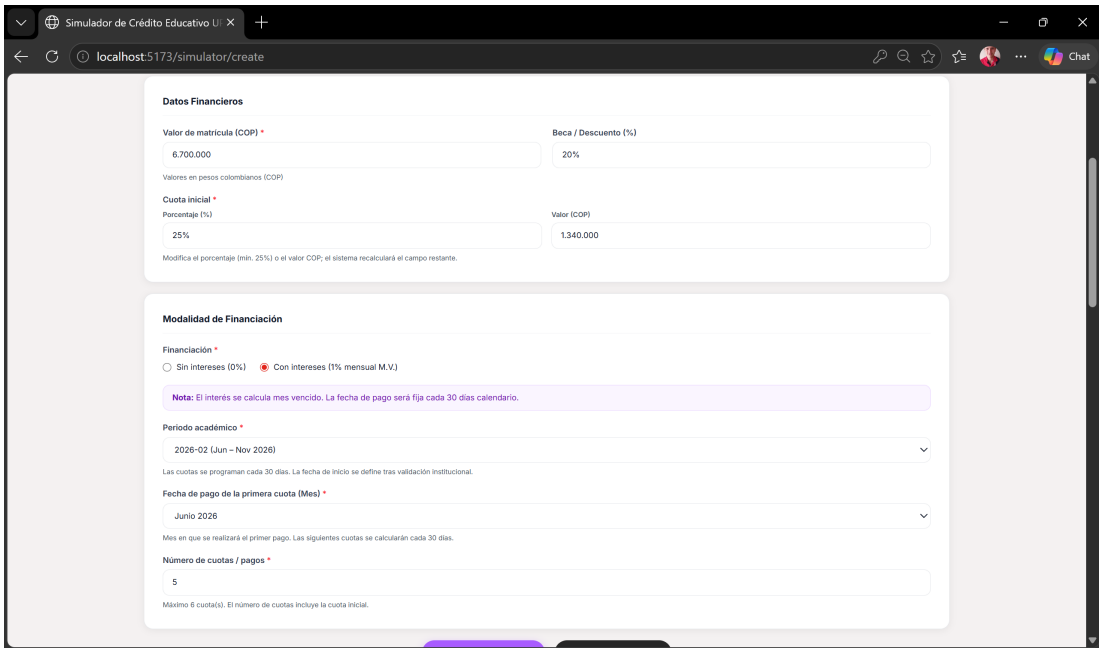
Motor financiero

El motor financiero es el núcleo funcional de la aplicación. Se implementa en Application/services/SimulationEngine.ts y utiliza estrategias intercambiables para calcular modalidades sin interés y con interés. La modalidad con interés aplica amortización francesa, mientras que la modalidad sin interés distribuye el valor financiado de forma lineal.

Regla	**Implementación / efecto**
UPB Tech ignora beca	Si educationLevel es upbTech, scholarshipPct se normaliza a 0.
Cuota inicial no es cuota del plan	El plan de financiación usa numberOfPayments - 1 para amortización.
Cuotas cortas sin interés	Si numberOfPayments <= 3 y llega con_interes, se fuerza sin_interes.
Fechas por calendario	Las fechas avanzan mes a mes con ajuste al último día válido.
Saldo final cero	La última fila absorbe residuo de redondeo para cerrar balance.
Parámetros dinámicos	Tasas, límites y porcentajes mínimos se leen desde versiones activas.

Figura 7

Resultado de simulación con interés y tabla financiera.



Las fórmulas presentadas corresponden a los dos métodos matemáticos utilizados por el motor del simulador para calcular los planes de pago. La primera ecuación representa el sistema de amortización francés, aplicado a las modalidades de crédito que generan intereses; esta operación toma el valor presente o monto a financiar y le aplica la tasa de interés periódica elevada al número total de periodos, logrando establecer una cuota fija mensual que cubre tanto el abono al capital como los intereses generados durante la vida del crédito.

$$cuota = PV * r * \frac{(1 + r)^n}{((1 + r)^n - 1)}$$

Por otro lado, la segunda ecuación ilustra el cálculo para planes de financiamiento sin intereses, donde el valor total financiado simplemente se divide de forma directa y equitativa entre el número de cuotas estipuladas en el plan, dando como resultado pagos fijos que abonan exclusivamente al capital neto de la deuda.

$$cuota = \frac{valor_{financiado}}{numero_{de_{cuotas_{del_{plan}}}}}$$

Modelo de datos

La persistencia usa MySQL/MariaDB con Sequelize. El esquema consolidado de CTIC se encuentra en docs/database/hu-66/scripts/00_ctic_full_deploy_mariadb_10_11.sql. Las tablas principales cubren usuarios, versiones de parámetros, reglas de cuota inicial, configuraciones por nivel, tasas, simulaciones, filas de amortización, auditoría, exportaciones y tokens de recuperación.

Figura 8

Diagrama de base de datos del simulador.

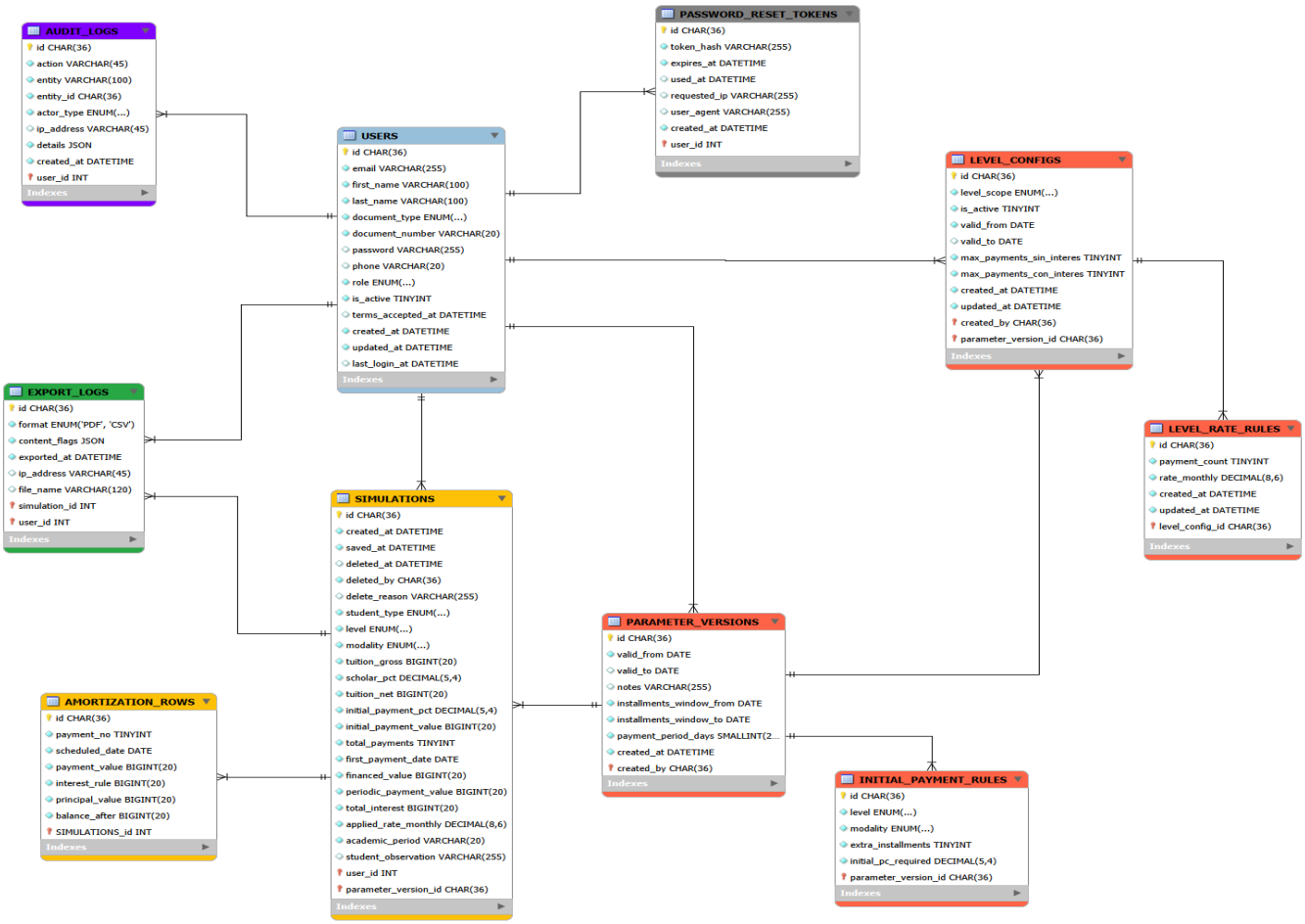


Tabla	Propósito
users	Usuarios estudiante y administrador, credenciales locales, documento, rol y estado.
parameter_versions	Versiones de parámetros financieros con vigencia.

initial_payment_rules	Reglas de cuota inicial por nivel, modalidad y número de cuotas.
level_configs	Configuraciones activas por nivel educativo.
level_rate_rules	Tasas mensuales por cantidad de pagos.
simulations	Resumen persistido de cada simulación guardada.
amortization_rows	Detalle de cada cuota de una simulación.
audit_logs	Registro de acciones de usuarios y sistema.
export_logs	Trazabilidad de exportaciones PDF/CSV.
password_reset_tokens	Tokens hash para recuperación de contraseña.

API y contratos

La API agrupa rutas públicas, rutas protegidas para estudiantes y rutas de administración. Los contratos se validan con DTOs Zod en Application/dtos y se exponen parcialmente en Swagger. Express Router permite organizar rutas como módulos montables (Express, n.d.).

Grupo	**Rutas representativas**	**Seguridad**
Auth	/api/auth/login, /register, /forgot-password, /reset-password	Rate limit; validación Zod; bcrypt/JWT.
Simulator público	/api/simulator/preview	Rate limit; validación y cálculo sin persistencia.
Simulator estudiante	/api/simulator, /history, /export/pdf, /export/csv	JWT STUDENT; ownership por usuario.
Users	/api/users/me	JWT; edición de perfil propio.
Admin	/api/admin/users, /simulations, /parameters	JWT + roleGuard ADMIN.
Parameters públicos	/api/parameters/active	Público para configurar formulario.

Seguridad

El modelo de seguridad sigue un enfoque de defensa en profundidad alineado con riesgos comunes de aplicaciones web documentados por OWASP (2021). El servidor aplica headers con Helmet, CORS, sanitización XSS, rate limiting, autenticación JWT, autorización por rol, validación Zod y manejo centralizado de errores.

Capa	**Implementación**	**Riesgo mitigado**
Headers	helmet()	Clickjacking, sniffing, políticas HTTP débiles.
CORS	cors()	Consumo desde orígenes no previstos.
Rate limit	authLimiter / simulatorLimiter	Fuerza bruta y abuso de endpoints.
Sanitización	xssMiddleware()	Entrada HTML/JS maliciosa.
Autenticación	authGuard + JwtTokenService	Acceso sin token o token inválido.
Autorización	roleGuard(UserRole.ADMIN)	Acceso a módulos no permitidos.
Validación	Zod schemas	Payloads inválidos o inconsistentes.
Auditoría	AuditService / audit_logs	Falta de trazabilidad operativa.

Cosas críticas a tener en cuenta
No versionar .env, certificados ni credenciales; rotar secretos si fueron expuestos; no publicar el puerto 3000; validar DB_SSL y ca.pem; usar HTTPS en producción; revisar logs antes de compartir evidencias.

Ambientes y variables

La configuración se gestiona con .env.example como plantilla y validación fail-fast en server/src/Infrastructure/config/env.ts. En producción, variables como LDAP, SMTP, DB y JWT son obligatorias. El frontend solo debe consumir variables con prefijo VITE, especialmente VITE_API_URL.

Variable	**Tipo**	**Notas**
NODE_ENV	backend	development, test o production.
PORT	backend	Puerto interno, por defecto 3000.
FRONTEND_URL	backend	URL pública usada en correos.
VITE_API_URL	frontend	Recomendado /api en producción.
DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD	BD	Conexión MySQL/MariaDB.
DB_SSL	BD	Si true en producción requiere server/certs/ca.pem.
LDAP_URL, LDAP_DOMAIN, LDAP_BASE_DN	auth	Autenticación institucional admin.
JWT_SECRET, JWT_EXPIRES_IN	auth	Firma y expiración de tokens.
SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS	correo	Recuperación de contraseña.
RETENTION_DAYS y TOKEN_PURGE_*	operación	Purga automática de simulaciones y tokens.

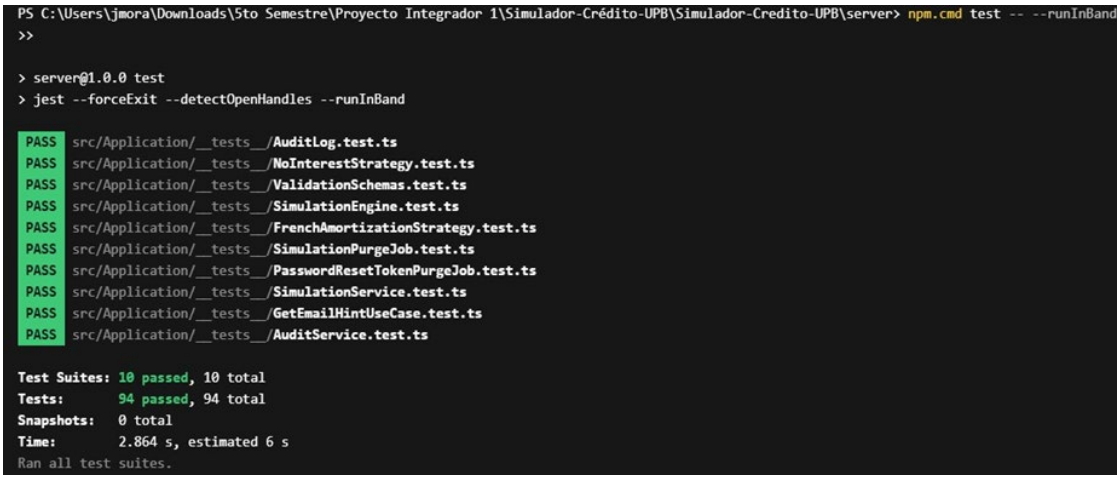
Pruebas y calidad

El proyecto conserva evidencias unitarias, de integración API, funcionales/manuales y de regresión. Para HU-56 se reportan 10 suites y 94 pruebas backend aprobadas. Para HU-57 se consolidan 47/47 casos aprobados entre integración API, funcional/manual y regresión, con 0 fallos y 0 bloqueos.

Bloque	Resultado	Evidencia
HU-56 unitarias	10 suites, 94 tests, 0 fallos	docs/architecture/unit-test.md
HU-57 integración	47/47 PASS, 0 FAIL, 0 BLOCKED	docs/architecture/integration-test.md
Frontend unitario	ProtectedRoute y FullPageSpinner	client/src/shared/components/__tests__
Backend unitario	Engine, estrategias, servicios, Zod y jobs	server/src/Application/__tests__
Documentación de evidencia	Capturas, JSON, CSV/PDF exportados	docs/architecture/assets

Figura 9

Evidencia consolidada de pruebas unitarias del backend.




```
# Backend
npm run test --workspace=server
```

```
npm run test:coverage --workspace=server

# Frontend
npm run test --workspace=client

# Build completo
npm run build --workspace=client
npm run build --workspace=server
```

Despliegue y operación

El despliegue productivo recomendado para CTIC usa una VM RHEL, Nginx como reverse proxy, PM2 para mantener el backend y MariaDB/MySQL como base de datos. El manual específico de despliegue generado para CTIC se encuentra en docs/deployment/Manual_Despliegue_CTIC.docx.

Componente	Ruta / puerto	Operación
Frontend build	client/dist -> /usr/share/nginx/html/simulador	Publicado como archivos estáticos.
Backend	localhost:3000	Ejecutado con PM2 desde server/dist/index.js.
Nginx	80/443	Sirve SPA y enruta /api al backend.
PM2	simulador-backend	Cluster, autorestart, pm2 save y systemd.
Logs	server/logs, pm2 logs, /var/log/nginx	Diagnóstico operativo.

```
cd /opt/simulador/app
npm ci
npm run build --workspace=client
npm run build --workspace=server
./server/scripts/deploy-rhel.sh
APP_DOMAIN=DOMINIO_PUBLICO PUBLIC_URL_SCHEME=http
./server/scripts/configure-nginx-rhel.sh
pm2 status
curl --fail http://localhost/api/health
```

Mantenimiento

Actividad	**Frecuencia sugerida**	**Responsable**
Revisar logs de PM2 y Nginx	Semanal o ante incidente	CTIC / soporte técnico
Backup de base de datos	Diario o según política CTIC	CTIC
Actualizar parámetros financieros	Cada periodo académico	Administrador funcional
Rotar secretos	Ante exposición o política institucional	CTIC
Ejecutar pruebas antes de release	Cada cambio	Equipo de desarrollo
Revisar dependencias vulnerables	Mensual o por release	Equipo de desarrollo / CTIC
Validar SMTP/LDAP	Antes de producción y ante cambios de red	CTIC

Riesgos y recomendaciones

Riesgo / deuda técnica	Impacto	Recomendación
Sin migraciones versionadas completas	Riesgo al crear o actualizar esquema productivo	Mantener scripts SQL controlados y evaluar migraciones formales.
Dependencia LDAP institucional	Login admin puede fallar por red o DN	Validar conectividad y monitorear tiempos de respuesta.
SMTP externo	Recuperación de contraseña queda inoperante si falla SMTP	Monitorear envío y configurar cuenta institucional estable.
Secretos en archivos locales	Exposición de credenciales	Usar gestor de secretos o permisos estrictos chmod 600.
Cobertura frontend limitada	Regresiones visuales no detectadas	Agregar tests de formularios y flujos críticos.
Swagger público	Exposición de superficie API	Decidir si /api/docs queda público o restringido en producción.

Recomendación de continuidad
Antes de entregar definitivamente a CTIC, generar un paquete limpio sin .git, .env, logs ni archivos de despliegue con secretos. Mantener este manual, el manual de despliegue y el script SQL de base de datos como paquete documental controlado.

Glosario

Término	Definición
SPA	Single Page Application; aplicación web que navega sin recargar páginas completas.
JWT	Token firmado usado para autenticar solicitudes HTTP.
LDAP	Protocolo usado para autenticación contra directorio institucional.
ORM	Mapeo objeto-relacional; en el proyecto se usa Sequelize.
PM2	Gestor de procesos Node.js para producción.
Reverse proxy	Servidor web que recibe tráfico externo y lo reenvía al backend interno.
Soft-delete	Eliminación lógica mediante campo deleted_at en lugar de borrado físico inmediato.
Seed	Carga inicial de datos mínimos como parámetros y usuario administrador.

Referencias

- Express. (n.d.). Express routing. https://expressjs.com/en/guide/routing.html Jest. (n.d.).
- Jest. (n.d.). Getting started. https://jestjs.io/docs/30.0/getting-started
- MariaDB Foundation. (n.d.). GRANT. https://mariadb.com/docs/server/reference/sql-statements/account-management-sql-statements/grant
- Meta Open Source. (n.d.). React reference overview. https://react.dev/reference/react
- Microsoft. (n.d.). TypeScript documentation. https://www.typescriptlang.org/docs/
- Nginx. (n.d.). nginx documentation. https://nginx.org/en
- Node.js Contributors. (n.d.). Node.js documentation. https://nodejs.org/docs/latest/api/
- Nodemailer. (n.d.). Nodemailer documentation. https://nodemailer.com/
- npm. (n.d.). Workspaces. https://docs.npmjs.com/cli/v11/using-npm/workspaces/
- Oracle. (n.d.). MySQL 8.4 Reference Manual. https://dev.mysql.com/doc/refman/8.4/en/
- Open Worldwide Application Security Project. (2021). OWASP Top 10:2021. https://owasp.org/Top10/2021/
- PDFKit. (n.d.). Getting Started with PDFKit. https://pdfkit.org/docs/getting_started.html
- PM2. (n.d.). Ecosystem File. https://pm2.io/docs/runtime/reference/ecosystem-file/
- React Hook Form. (n.d.). Documentation. https://react-hook-form.com/docs
- React Router. (n.d.). React Router Home. https://reactrouter.com/home
- Sequelize. (n.d.). Sequelize v6. https://sequelize.org/docs/v6/
- Tailwind Labs. (n.d.). Installing Tailwind CSS with Vite. https://tailwindcss.com/docs/installation/using-vite
- Vite. (n.d.). Getting Started. https://vite.dev/guide/
- Vitest. (n.d.). Getting Started. https://main.vitest.dev/guide/
- Zod. (n.d.). Intro. https://zod.dev/

- Equipo de desarrollo del Simulador de Crédito UPB. (2026). Documentación técnica interna del repositorio: README, architecture, deployment, database y testing.